



Context Budget Allocation : Research Review & Benchmark Plan



Research module: deterministic context-budget allocation for agentic AI systems.

Meeting goal: present three credible approaches, show what Benchmark 1 actually measured, explain the surprising failures, and define the next controlled evaluation.

Presentation note: this version uses image blocks instead of Mermaid/code and avoids columns so it remains stable in Notion Present mode.

1 · The problem: too much context, finite budget



The allocator sits after tenant/auth filtering and context shaping. Its job is deterministic: decide which useful candidates survive when the available input budget is smaller than the authorized context.



Summarization, trimming, retrieval, and subagents reduce pressure, but they do not remove the final selection problem. The allocator

protects system/security instructions, reserves output and tool headroom, then distributes the remaining elastic budget.

2 · Three approaches studied

A · Fixed quota + spillover

Reserve a fixed share for each elastic class, then redistribute unused quota deterministically.

Example shares: history 30% · RAG 25% · tools 20% · agents 15% · memory 10%

Why it is credible: simple to audit, cheap to run, predictable under pressure.

Main risk: static shares may not match the workload shape.

B · Floors + weighted water-fill

Protect mandatory context, guarantee minimum floors, distribute leftover capacity by class weights, and enforce ceilings.

Flow: protected lane → floors → weighted leftover → ceilings → item packing

Why it is credible: adaptive to workload shape and explicit about starvation safeguards.

Main risk: ceilings and within-class ordering can dominate the result.

C · Constrained utility packing

Protect mandatory context and minimums, then let individual items compete using an explicit utility score under the token budget.

Approximate score: relevance × class weight, with recency used where a native relevance score is unavailable.

Why it is credible: can preserve small, high-value evidence over large, low-value context.

Main risk: utility scores across heterogeneous classes are difficult to define and calibrate.



None of the three is selected as the production algorithm yet.
Benchmark 1 is designed to challenge the water-fill hypothesis, not to

prove it.

3 · Benchmark 1: a controlled selection experiment

Dimension	Benchmark 1 setup	What it isolates
Workloads	16 synthetic workload shapes	normal pressure, mixed overflow, failure probes, security pressure
Windows	32k · 64k · 128k · 256k	tight to generous input budgets
Policies	fixed quota · water-fill · utility · recency baseline	three candidates plus a harness comparator
Total	$16 \times 4 \times 4 = 256$ cells	same workloads and accounting for every policy

Primary structural metric: critical-evidence recall — whether the required answer-bearing item survived allocation.

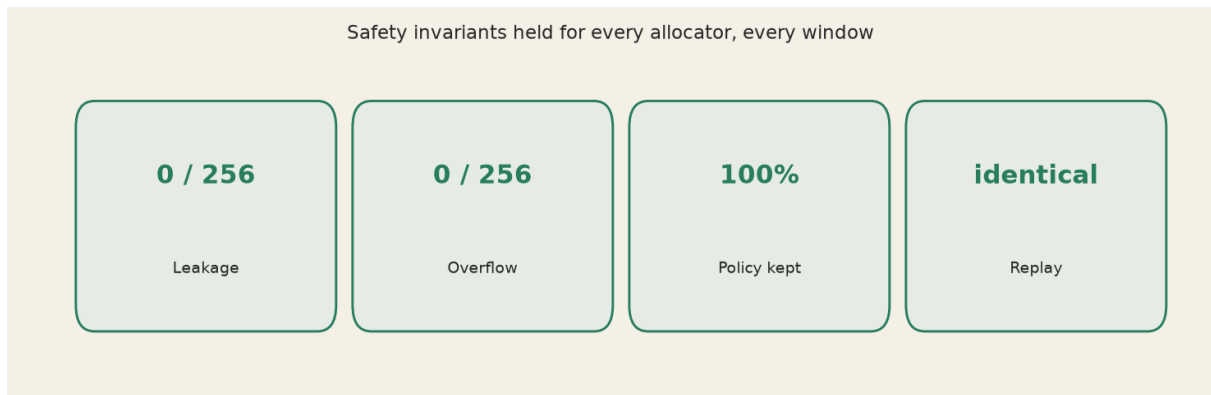
Other metrics: protected-policy retention · leakage · overflow · starvation · utilization · runtime · tokenizer mismatch · deterministic prompt/selection hashes.

Ground-truth answer IDs were kept separate from allocator-visible metadata, so the allocator could not be told which item contained the answer.

Input budgets after reserves

Physical window	Allocator input budget	Reserved capacity
32k	28,160	output + tool-call headroom
64k	56,320	output + tool-call headroom
128k	120,832	output + tool-call headroom
256k	243,712	output + tool-call headroom

4 · Shared safety envelope



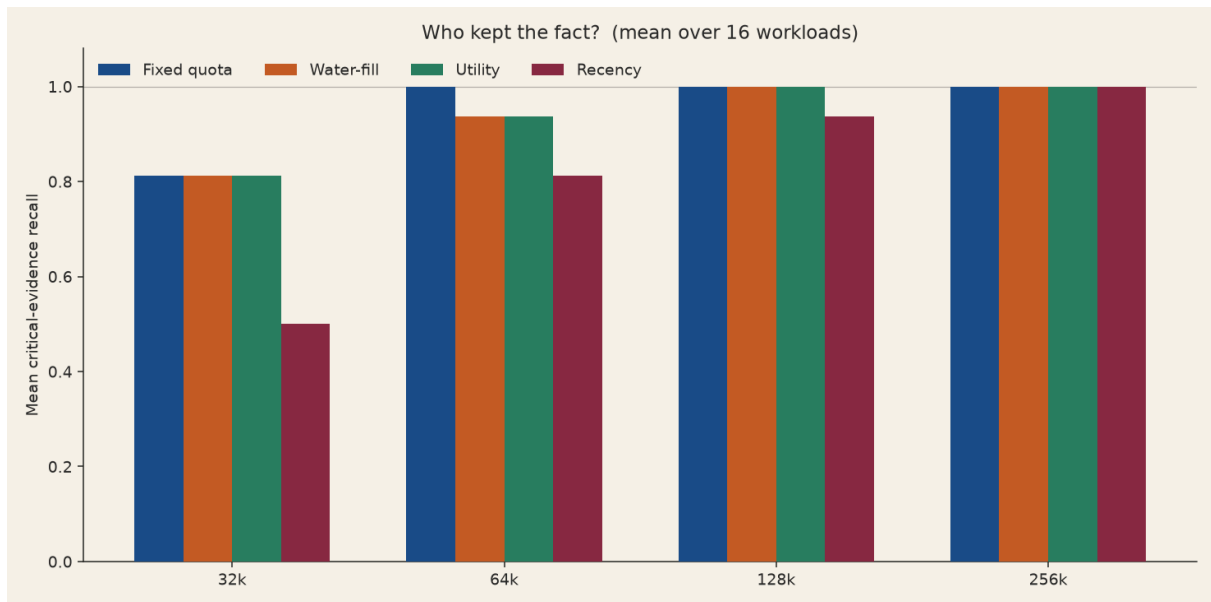
Every policy used the same non-negotiable envelope:

- tenant/auth filtering before allocation
- protected system and security context
- reserved output and tool-call capacity
- exact tokenizer accounting
- deterministic tie-breaks and assembly
- fail-closed behavior for protected overflow

Benchmark 1 safety result: leakage 0 / 256 · overflow 0 / 256 · protected policy kept 100% · replay identical.

This means the comparison is about elastic allocation quality, not which policy forgot basic security.

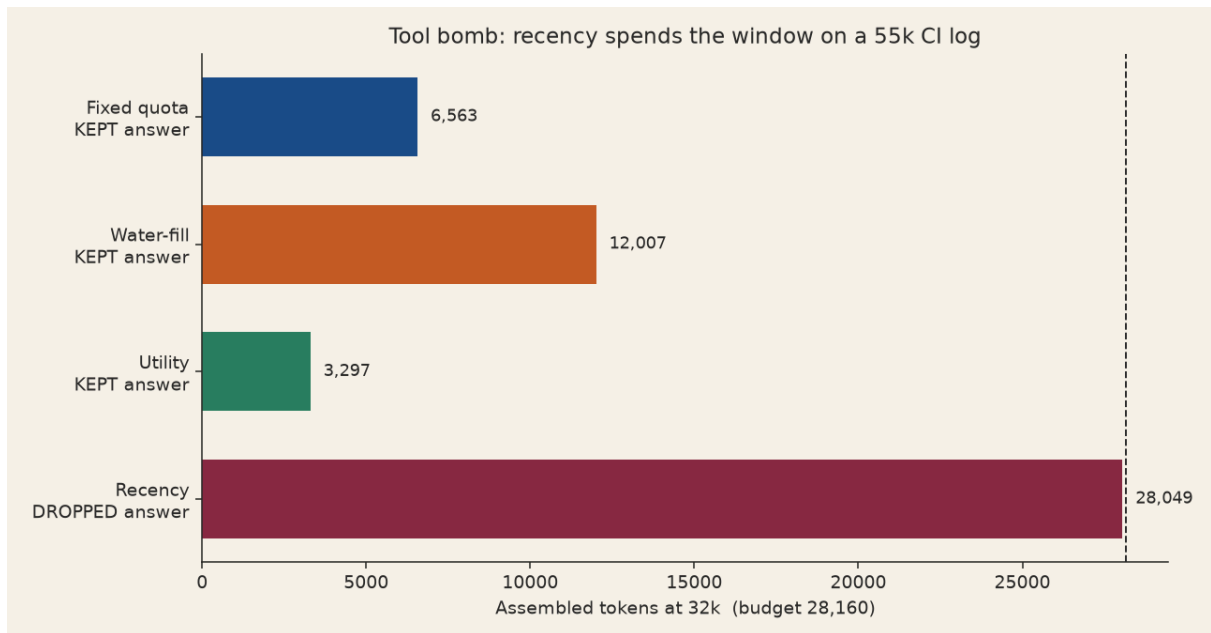
5 · Result: no unique winner yet



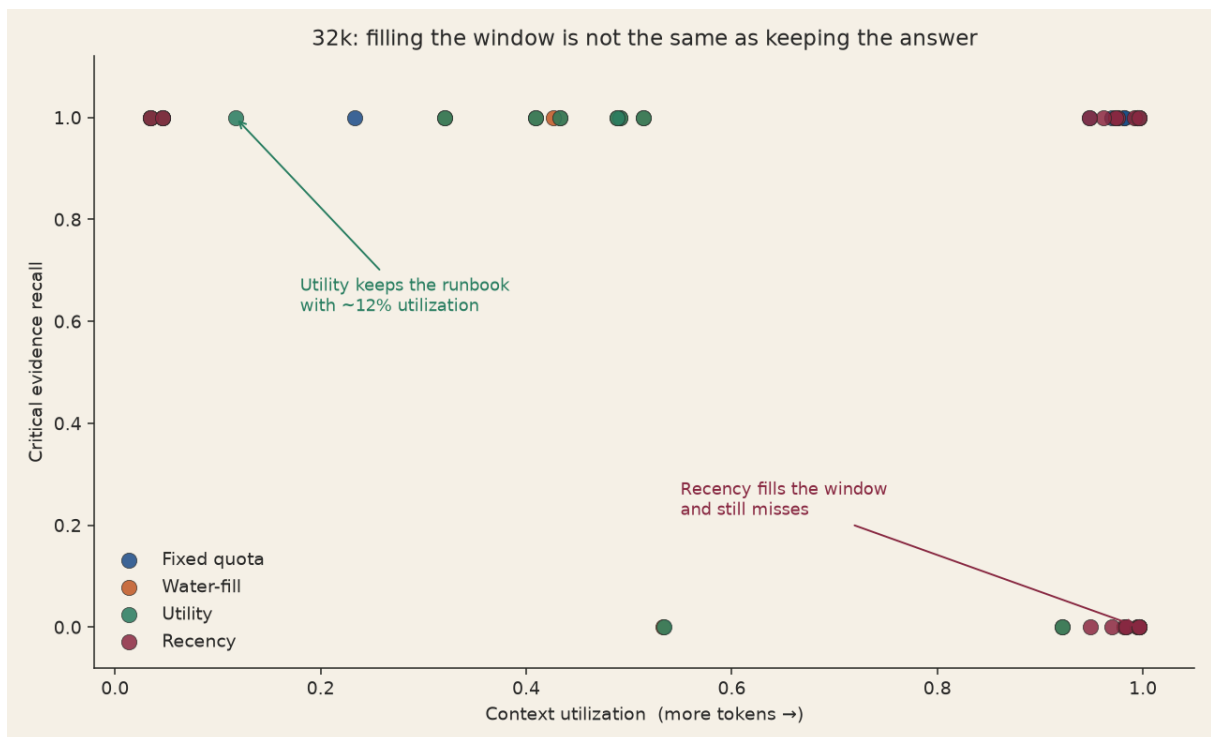
Input window	Fixed quota	Water-fill	Utility	Recency
32k	81.2%	81.2%	81.2%	50.0%
64k	100%	93.8%	93.8%	81.2%
128k	100%	100%	100%	93.8%
256k	100%	100%	100%	100%

Interpretation: fixed quota wins at 64k because spillover reaches an older important tool result that water-fill and utility miss after hitting a 40% tool ceiling with newest-first packing. The surprise is caused by the interaction between ceiling policy and within-class ordering, not by water-fill arithmetic alone.

6 · Finding 1: filling the window is not the objective



A recent 55k CI log can consume nearly the entire 32k input budget under pure recency. It reaches about 99.6% utilization but drops the tiny runbook containing the answer.



Fixed quota, water-fill, and utility retain the answer with far fewer assembled tokens. Utility keeps it at roughly 12% utilization.



Maximum context utilization is not maximum task quality. The allocator should preserve useful evidence, not simply maximize the number of tokens sent.

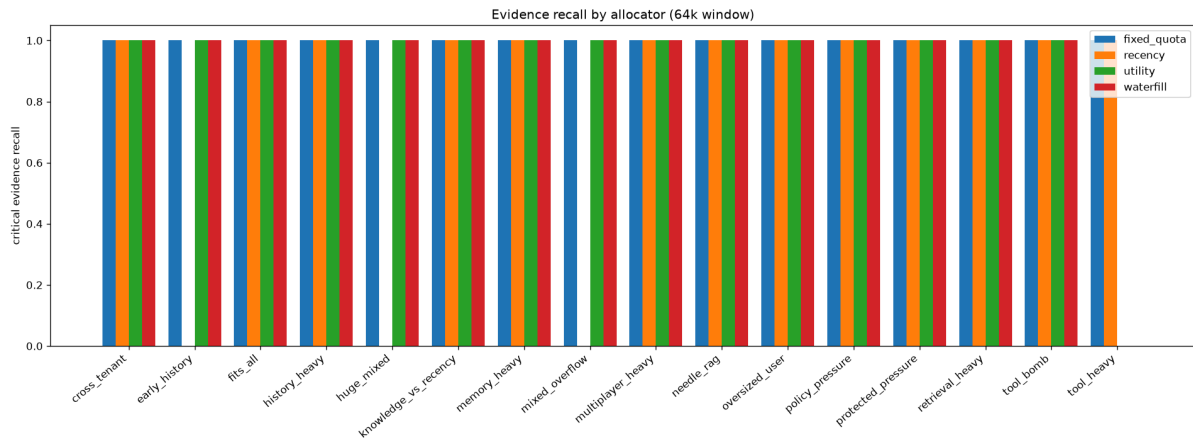
7 · Finding 2: the decisive variables are below the top-level allocator

32k workload detail

32k window — green kept the fact, red dropped it				
cross_tenant	kept	kept	kept	kept
early_history	kept	kept	kept	miss
fits_all	kept	kept	kept	kept
history_heavy	kept	kept	kept	kept
huge_mixed	kept	kept	kept	miss
knowledge_vs_recency	kept	kept	kept	kept
memory_heavy	kept	kept	kept	kept
mixed_overflow	kept	kept	kept	miss
multiplayer_heavy	kept	kept	kept	miss
needle_rag	miss	miss	miss	miss
oversized_user	miss	miss	miss	miss
policy_pressure	kept	kept	kept	kept
protected_pressure	kept	kept	kept	kept
retrieval_heavy	kept	kept	kept	kept
tool_bomb	kept	kept	kept	miss
tool_heavy	miss	miss	miss	miss
	Fixed quota	Water-fill	Utility	Recency

At 32k, all policies share hard cases such as `needle_rag`, `oversized_user`, and `tool_heavy`. Recency adds several workload-specific misses, including `tool_bomb`, `mixed_overflow`, and `early_history`.

64k workload detail



At 64k, most workload profiles pass. The remaining aggregate gap is concentrated in a small number of workload-specific misses, which is why Benchmark 2 needs controlled ablations rather than a single overall score.



Key research insight: cross-class allocation is only one layer. Within-class packing, ceiling behavior, and current-user truncation can change the outcome as much as quota versus water-fill.

8 · What Benchmark 1 supports — and what it does not

Supported so far

- a privileged system/security lane is necessary
- tenant filtering must happen before allocation
- output and tool headroom must be reserved first
- deterministic reconstruction and replay are achievable
- pure recency is fragile under heterogeneous pressure
- utilization alone is a poor objective
- fixed quota, water-fill, and utility converge at larger windows

Not proven yet

- that water-fill is strictly better than quotas

- any particular floor, weight, or ceiling value
 - that retained evidence is actually used by the model
 - production-tokenizer safety for a serving model
 - that the synthetic workload distribution matches real traffic
-

9 · Benchmark 2: isolate the actual causes

Keep the safety envelope fixed and change one policy variable at a time.

1. **Cross-class allocation:** fixed quota versus water-fill versus utility.
2. **Within-class packing:** newest-only versus first-plus-recent versus score-based selection.
3. **Ceilings:** hard headroom versus donating unused capacity.
4. **Current-user cap:** unbounded input versus bounded head-plus-tail handling.
5. **Sensitivity:** sweep floors, weights, and ceilings instead of treating one configuration as truth.
6. **Model-use evaluation:** send the assembled contexts to the same model and compare evidence present versus answer correct.
7. **Tokenizer validation:** rerun with the real serving-model tokenizer when known.

Budget sweep: 32k · 64k · 128k · 256k

10 · Current working position



Do not choose an allocator yet. The leading architecture is a deterministic planner with an authorization/protected lane and adaptive elastic allocation. The first pass says the final design may be a combination of cross-class policy, within-class selection, and ceiling/headroom behavior.

The decision should come after Benchmark 2 identifies which policy retains useful context most reliably under tight budgets without compromising safety,

replay, latency, or cost.

summary

Three approaches: fixed quota + spillover · weighted water-fill · constrained utility packing

Benchmark 1: 16 workloads × 4 windows × 4 policies = 256 deterministic cells, with structural evidence metrics and shared safety invariants.

Current evidence: no unique winner yet; pure recency is fragile; fixed quota, water-fill, and utility converge at larger windows; within-class packing and ceilings are now the highest-value variables to isolate.

Next: controlled ablations first, then model-level answer-quality evaluation.